

Trace-based Uncertainty Estimation for Reasoning Language Models

Dagmawi Misker Gedamu (dagmawi@aims.ac.za)
African Institute for Mathematical Sciences (AIMS)

Supervised by: Dr. Umang Bhatt
Cambridge University, UK

20 June 2026

Submitted in partial fulfillment of a structured masters degree at AIMS South Africa



Abstract

Reasoning language models achieve strong performance on complex tasks that require multi-step logical reasoning, such as mathematics and programming. Despite these advances, they remain susceptible to hallucinations and failures to abstain, limiting their reliability in practice. Uncertainty quantification (UQ) can help address these issues; however, existing approaches typically rely on signals derived only from the final answer. This thesis investigates whether the structural and textual properties of internal reasoning traces can be used to estimate the uncertainty of the model. We extract lightweight trace-based features and use them to predict answer correctness through a simple classifier. Experiments across multiple benchmarks show that the proposed approach consistently outperforms standard self-evaluation methods, including verbalised confidence and $P(\text{True})$. On multiple-choice reasoning tasks, it also surpasses semantic entropy when applied to capable reasoning models. Furthermore, we demonstrate that an effective single-pass variant can be obtained from a single greedy trace, reducing computational cost while maintaining competitive performance.

Declaration

I, the undersigned, hereby declare that the work contained in this research project is my original work, and that any work done by others or by myself previously has been acknowledged and referenced accordingly.



Dagmawi Misker Gedamu, 20/06/2026

Contents

Abstract	i
1 Introduction	1
1.1 Problem Statement	1
1.2 Objectives	1
1.3 Structure of the Thesis	1
2 Literature Review	2
2.1 Large Language Models and Their Reliability	2
2.2 Reasoning in Large Language Models	2
2.3 Uncertainty in Machine Learning and LLMs	3
2.4 Uncertainty Quantification Methods	4
2.5 Hallucination and the Failure to Abstain	7
2.6 Summary and Research Gap	8
3 Methodology	9
3.1 Data Generation	9
3.2 Feature Engineering and Extraction	10
3.3 Analysis	12
3.4 Experimental Setup	14
4 Results	17
4.1 Data and Exploratory Analysis	17
4.2 Feature Analysis	18
4.3 The trace_LR Feature Set	21
4.4 Comparison with Baseline Methods	21
4.5 Greedy versus Sampled Trace Features	22
4.6 Combining Trace Features with Semantic Entropy	24
4.7 Calibration	26
4.8 Discussion	26
5 Conclusion and Future Work	28
5.1 Limitations	28
5.2 Future Work	28
References	33
Appendices	34

1. Introduction

1.1 Problem Statement

Large language models are susceptible to hallucination, a phenomenon in which they give semantically plausible but factually incorrect information (Huang et al., 2023). This issue becomes critical when integrating these models into a pipeline that requires autonomous judgements and decisions. Reasoning models enhance performance by decomposing complex problems into sequential steps and generating multi-step reasoning chains prior to producing a final answer (OpenAI, 2024).

However, they remain susceptible to the same problems as non-reasoning architectures, and this vulnerability is magnified in specific cases such as abstention (Kirichenko et al., 2025; Mei et al., 2025). Many uncertainty quantification methods have been developed with non-reasoning models in mind and measure uncertainty solely using the final answer (Kuhn et al., 2023; Tian et al., 2023). These methods do not account for the information generated by the model during the reasoning process, which could be a strong signal of its uncertainty.

1.2 Objectives

This study aims to develop an uncertainty quantification method for reasoning models that is based on the properties of the reasoning traces they produce while generating answers. More specifically, this research will:

- Analyse the traces of reasoning models and identify properties that can be used to measure uncertainty.
- Compare the performance of the new uncertainty quantification method with widely used baseline methods.
- Investigate the performance of the new uncertainty quantification method with non-reasoning models using a chain-of-thought prompt to determine if the effect stems from reasoning fine-tuning or simply from generating intermediate steps prior to a solution.

1.3 Structure of the Thesis

This thesis is organised as follows:

- **Chapter 2: Literature Review:** Reviews the literature on Large Language Models (LLMs), uncertainty quantification methods, and hallucination and the failure to abstain in LLMs.
- **Chapter 3: Methodology:** Describes the datasets and models used in the study, the experimental pipeline implemented to generate and analyse the data, and the construction of the evaluation metrics.
- **Chapter 4: Results and Discussion:** Presents the results of the study, discusses the findings, and explores the implications of these results.
- **Chapter 5: Conclusion:** Concludes the study, addresses certain limitations, and outlines opportunities for future work.

2. Literature Review

2.1 Large Language Models and Their Reliability

Large Language Models (LLMs) are built on the transformer architecture (Vaswani et al., 2017) and the models we study are decoder-only models that predict the next word from a probability distribution. Although the underlying mechanism is simple, training at scale on large corpora produces models that generate fluent text and perform a wide range of language tasks. Through fine-tuning and integration with external tools or retrieval systems, modern LLMs have moved well beyond plain text generation and are now used for multi-step problem solving and decision support (Wei et al., 2022).

This expanded capability has led to the use of LLMs in high-stakes fields where errors can cause serious problems, such as medical advice or financial decision support. Even in general knowledge applications, users expect answers they can rely on. Because these systems are increasingly used to help people make decisions, their reliability and trustworthiness are of critical importance.

Despite their high performance on standardised tests, including maths and reasoning (Anthropic, 2024; Google DeepMind, 2025), LLMs face major limitations in real-world environments. In practice, the inputs an LLM receives are rarely clean: a user's goal may only be partly specified, essential context might be missing from the prompt, or the model may encounter situations that exceed its knowledge or reasoning capabilities. Under such conditions, LLMs are prone to *hallucination*: generating content that is unsupported, unverifiable, or inconsistent with available information (Huang et al., 2023). A particular concern is that these errors are often made with high confidence, because the probabilities the model attaches to its own outputs are frequently poorly calibrated (Tian et al., 2023). A model that cannot recognise when it is unsure or when a question cannot be answered as posed is risky to rely on. This issue motivates the need for uncertainty quantification. Providing a system with a usable estimate of how much its response can be trusted is a basic requirement to deploy LLMs responsibly.

2.2 Reasoning in Large Language Models

Traditional LLMs perform reasoning only implicitly. Through next-token prediction they map the prompt to an answer in a single forward pass without writing out intermediate steps. This works for simple tasks, but it tends to fail on problems that require several steps of inference, such as multi-step arithmetic or logical deduction, where the answer depends on a chain of intermediate results. Two broad approaches have emerged to make models reason more explicitly, both of which spend additional computation at inference time.

The first approach is *chain-of-thought (CoT) prompting* (Wei et al., 2022; Kojima et al., 2022). This is a straightforward method that modifies the prompt to instruct the model to produce step-by-step explanations before giving its final answer. Kojima et al. (2022) shows that even appending a simple instruction such as "Let's think step by step" substantially improves performance on arithmetic and reasoning tasks, without any further training. This technique does not change the model; it changes what the model is asked to produce.

The second approach produces *reasoning models* (also called thinking models) through additional training rather than prompting. This paradigm was introduced by OpenAI's o1 models (OpenAI, 2024) which were trained to perform extended reasoning at inference time and demonstrated large gains on hard reasoning benchmarks. Open models such as DeepSeek-R1 (DeepSeek-AI, 2025) subsequently showed that

similar reasoning behaviour could be elicited through reinforcement learning, and are trained to generate an extended internal reasoning trace before committing to a final answer. The training rewards reaching correct solutions, which encourages behaviours such as exploring alternatives, checking intermediate steps, and self-correcting within the trace. Because the reasoning trace is produced as part of the output, often delimited by explicit markers such as `<think>` and `</think>`, the model's intermediate steps are visible, which is part of what makes reasoning models attractive for tasks where the working matters.

These gains come with new problems. Reasoning models are prone to *overthinking*: on simple questions they may generate very long traces that add cost without improving, and sometimes harm, accuracy (Cuadron et al., 2025). More importantly for reliability, recent work shows that reasoning models are often poor at recognising their own limits. Kirichenko et al. (2025) find that reasoning models frequently fail to recognise unanswerable questions and instead produce confident answers, and that intensive reasoning fine-tuning can even worsen this behaviour. Mei et al. (2025) examine whether reasoning models know when they do not know, and report that they are systematically overconfident, with calibration worsening as the amount of reasoning increases: rather than resolving uncertainty, longer reasoning can entrench it. Consequently, models that excel at complex reasoning can display high overconfidence when evaluating tasks outside their training domain; this tendency highlights the clear value of reliable uncertainty estimates.

2.3 Uncertainty in Machine Learning and LLMs

Quantifying uncertainty is a long-standing concern in machine learning, studied for classifiers and neural networks well before large language models. A model that returns only a prediction, with no indication of how reliable that prediction is, leaves the user unable to tell a well-grounded answer from a guess, which matters most precisely when the prediction informs a decision. A useful first step is to ask where a model's uncertainty comes from, since the total (predictive) uncertainty of a model is commonly split into two components (Hüllermeier and Waegeman, 2021).

Aleatoric uncertainty is the irreducible part, arising from genuine randomness or ambiguity in the data itself. No amount of extra training removes it. For an LLM, an example is a question whose answer is inherently underdetermined, such as “What will the temperature be tomorrow?”, or an ambiguous prompt that admits several valid readings (Ahdritz et al., 2024).

Epistemic uncertainty is the reducible part, arising from the model's own lack of knowledge, for instance, because the relevant fact was rare or absent in training. In principle, it can be reduced by better or more data. When a model is asked for a fact it never learned, the resulting uncertainty is epistemic.

This distinction matters because the two call for different responses: epistemic uncertainty signals a knowledge gap, whereas aleatoric uncertainty signals that the question itself is ambiguous or unanswerable, which is exactly the case where a model ought to abstain rather than guess. Some recent work has tried to decompose an LLM's uncertainty into these components, for example by ensembling predictions over clarified versions of an ambiguous prompt (Hou et al., 2024), although others argue that the classical split is hard to apply cleanly to open-ended language tasks (Ahdritz et al., 2024). For our purposes, we do not attempt to separate the two. We measure uncertainty as a single predictive signal and ask whether it indicates that an answer is likely to be wrong.

2.4 Uncertainty Quantification Methods

Given these notions of uncertainty, the question becomes how to measure them in practice. In deep learning more broadly, this has been tackled with general-purpose methods such as Bayesian neural networks, which learn a distribution over the model's weights rather than a single point estimate and treat the spread of that distribution as uncertainty, along with their more tractable approximations such as Monte-Carlo dropout and deep ensembles, which train several models and read off disagreement between them as a measure of uncertainty (Shorinwa et al., 2025).

These methods, however, transfer poorly to large language models. Running inference on an LLM is expensive, so techniques that require training many models or sampling many full forward passes, as ensembles and Bayesian methods do, become impractical at this scale. Just as importantly, the methods were designed for fixed-size classification or regression outputs, whereas an LLM produces a variable-length sequence of tokens, and the “answer” is a piece of free-form text that can be phrased in many equivalent ways. These differences, the cost of inference and the open-ended, sequential, paraphrase-rich nature of the output, are why a distinct family of UQ methods has been developed specifically for LLMs.

The LLM-specific methods are commonly organised by what information they require. *White-box* methods read signals from inside the generation process, mainly the probabilities the model assigns to its tokens, and sometimes its internal states. *Black-box* methods use only the model's text outputs, which makes them applicable even to closed models accessed through an interface.

2.4.1 White-Box Methods

White-box methods read the probabilities the model assigns to tokens as it generates, so they require access to the output distribution (the logits or log-probabilities). For a generated sequence $y = (y_1, \dots, y_T)$ given input x , the standard quantities are the following.

Mean token log-probability averages the log-probability of the generated tokens, providing a simple measure of the model's average confidence in the tokens it produced, as shown in 2.4.1:

$$\overline{\log p} = \frac{1}{T} \sum_{t=1}^T \log p_{\theta}(y_t \mid x, y_{<t}). \quad (2.4.1)$$

More negative values indicate lower confidence, whereas values closer to zero indicate higher confidence.

Predictive entropy measures the uncertainty of the model's output distribution. For the whole sequence it can be written in terms of the conditional distribution $p_{\theta}(y \mid x)$ as

$$H(Y \mid x) = - \sum_y p_{\theta}(y \mid x) \log p_{\theta}(y \mid x), \quad (2.4.2)$$

and is typically approximated from sampled sequences or token-level entropies. Higher entropy means the model spreads probability over many alternatives, indicating greater uncertainty.

Perplexity is a closely related quantity, defined as the exponential of the average negative log-probability of the generated tokens, as shown in 2.4.3:

$$\text{PPL}(y) = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(y_t \mid x, y_{<t})\right), \quad (2.4.3)$$

A higher perplexity indicates that the model found its own output less probable and is therefore taken as a sign of greater uncertainty.

A further white-box family probes the model’s *internal states* rather than its output probabilities. Methods such as SAPLMA (Azaria and Mitchell, 2023), which trains a classifier on hidden-layer activations to predict whether a statement is true, and internal-state hallucination detectors such as MIND (Su et al., 2024), fall here. These require access to activations and are therefore the most demanding in terms of model access.

2.4.2 Black-Box Methods

Black-box methods avoid the need for internal probabilities or states, which makes them suitable for closed models and is the setting we adopt in this work. The three we use as baselines are described below.

Semantic entropy (Kuhn et al., 2023; Farquhar et al., 2024) addresses a problem specific to language: the same answer can be phrased in many different ways, so measuring entropy over raw token sequences confuses surface variation with genuine disagreement. The key idea is to generate several answers to the same question, group them by *meaning* rather than wording, and then compute the entropy over the resulting groups. Answers that mean the same thing are placed in one cluster, so the entropy reflects how much the model disagrees about the *content* of its answer, not how it phrased it. The clustering is usually done with a natural language inference model that tests whether two answers entail each other (Kuhn et al., 2023). If p_c is the fraction of samples in semantic cluster c , the semantic entropy is

$$H = - \sum_c p_c \log p_c. \quad (2.4.4)$$

Semantic entropy is one of the most widely used black-box methods and a strong baseline, but because it needs several samples per question it is comparatively expensive.

Verbalised confidence (Tian et al., 2023; Lin et al., 2022) simply asks the model to state how confident it is, in words. The confidence can be elicited categorically (for example, low / medium / high) or numerically (for example, a percentage from 0 to 100). While this method is computationally inexpensive and simple to interpret, its reliability varies because models often generate confidence levels that do not align with their actual accuracy.

P(True) (Kadavath et al., 2022) is a self-evaluation method in which the model judges the correctness of its own answer. After producing an answer, the model is re-prompted with the question and its proposed answer and asked whether the answer is true or false; the probability it assigns to the “True” token, normalised against the “False” token, is used as a confidence score:

$$P(\text{True}) = \frac{P(\text{“True”})}{P(\text{“True”}) + P(\text{“False”})}. \quad (2.4.5)$$

The intuition is that a model may carry information about whether its own output is correct, which can be surfaced by asking it to evaluate itself. Computing $P(\text{True})$ requires the output token probabilities for the True/False judgment, but not the model’s hidden states or weights. Its classification is debatable, because it does read token probabilities; following the convention used in much of the UQ literature

(Kadavath et al., 2022), we treat it as a black-box self-evaluation method, since it needs only the model’s output probabilities and no access to internal representations or parameters. A known limitation is that these self-judgements are often poorly calibrated, as models tend to be overconfident about their own answers.

2.4.3 Reasoning-Based UQ Methods

A smaller but growing line of work tries to extract uncertainty signals from the reasoning process itself rather than only from the final answer. The closest to ours is CoT-UQ (Zhang and Zhang, 2025), a response-wise framework that folds an LLM’s own chain-of-thought into its confidence estimate.

CoT-UQ works in two stages. In the first stage, the model is prompted to reason step by step, and the framework then processes that reasoning: it extracts the most meaningful keywords from each step (discarding redundant tokens that would otherwise add noise to the uncertainty score), and asks the model to rate, on a scale from one to ten, how important each keyword is for reaching the final answer. In the second stage, this reasoning information is fed back into otherwise standard UQ strategies.

For example, instead of reading the token probability of the final answer alone, an aggregated-probability variant computes an importance-weighted average over the probabilities of the keyword tokens, and a self-evaluation variant supplies the reasoning path as extra context when asking the model whether its own answer is correct. This approach is motivated by the observation that errors often hinge on a specific shaky step: a model may be highly confident overall yet only weakly confident in one critical fact in the chain, and surfacing that weak step yields a better-calibrated final confidence than scoring the answer in isolation.

CoT-UQ shares our central premise, that the reasoning trace carries information about the model’s confidence that the final answer does not. It differs from our approach in two ways that matter. First, it still relies on the model’s own token probabilities and self-evaluations, whereas we use lightweight, interpretable features computed directly from the trace text, namely its length, its repetition, and its hedging language. Second, CoT-UQ treats the trace as a source of keywords to re-weight existing probability-based scores, whereas we ask a different question: whether simple structural and lexical properties of the trace, on their own, predict correctness, and how that varies between reasoning and non-reasoning models and across answer formats.

2.4.4 Evaluating Uncertainty Estimates

Because no UQ method gives a “true” uncertainty, methods are judged by how useful their scores are. Three measures recur in this work and in the literature. *Discrimination* asks whether the score separates correct from incorrect answers, and is usually measured by the area under the ROC curve (AUROC) (Farquhar et al., 2024; Zhang and Zhang, 2025), which is threshold-free. *Calibration* asks whether a stated confidence matches the realised accuracy, and is commonly measured by the Expected Calibration Error (ECE) (Mei et al., 2025; Tian et al., 2023); ECE has known limitations, however, since it can be minimised by uninformative predictions that collapse towards the base rate, and is therefore often read alongside proper scoring rules such as the Brier score. *Selective prediction* asks how accuracy improves as the least-confident answers are withheld, summarised by the risk–coverage curve which is the setting most directly tied to abstention.

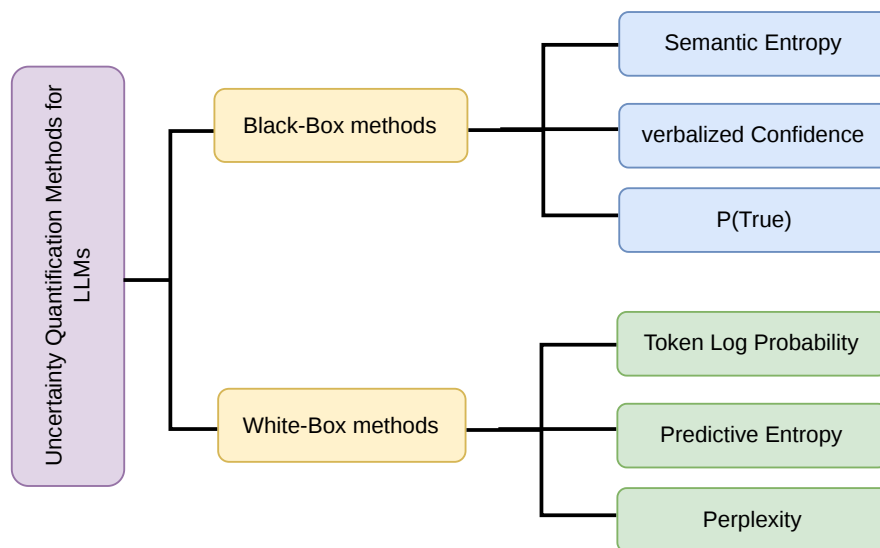


Figure 2.1: A taxonomy of uncertainty quantification methods for LLMs, organised by the access they require. Our trace-based approach sits in the black-box branch for reasoning models.

2.5 Hallucination and the Failure to Abstain

The practical reason to measure uncertainty is to avoid two linked failures: hallucinating, and failing to abstain when abstaining is the right choice.

Hallucination is the tendency of LLMs to produce answers that are fluent and plausible but factually wrong (Huang et al., 2023). It is rooted in the generative nature of these models, which are trained to produce likely continuations rather than to verify facts, and it is often accompanied by high confidence. Several authors frame hallucination partly as a symptom of a model’s inability to assess its own uncertainty: a model that knew it did not know would be less likely to fabricate.

Closely related is the problem of *abstention*: knowing when not to answer. A reliable model should recognise when a question cannot be answered as posed, for example when it is genuinely unanswerable, underspecified, or built on a false premise, and respond by clarifying, flagging the problem, or simply saying “I don’t know” rather than guessing. In practice, many models answer almost anything put to them, responding definitively even when they should decline (Wen et al., 2025). Using an uncertainty signal to decide when to abstain can improve safety and reduce hallucination (Tomani et al., 2024), yet off-the-shelf models still struggle to make this judgement on their own.

This failure is sharper, not milder, for reasoning models, which is initially surprising: one might expect a model that reasons step by step to be better at noticing when a question is unanswerable. Two recent studies show the opposite. (Kirichenko et al., 2025) build a large benchmark of unanswerable questions and find that reasoning fine-tuning actually *degrades* a model’s tendency to abstain, by around 24% on average relative to the same model before reasoning fine-tuning. This holds even in the mathematics and science domains where reasoning models are explicitly trained to excel: faced with a problem that is missing a needed quantity, the model often invents the missing information and pushes through to a confident final answer rather than pointing out that the question cannot be answered.

Two additional findings from that study are particularly important. First, giving the model a larger

“thinking budget” (more reasoning tokens) raises its accuracy on solvable problems but tends to make its abstention worse, not better. Second, and most directly relevant, the reasoning trace itself frequently does contain expressions of uncertainty, noting that information is missing or that a step is shaky, yet the model overrides this doubt and produces an assertive final answer.

A complementary study by (Mei et al., 2025) examines how well reasoning models’ stated confidence matches their actual accuracy. They find these models are systematically overconfident, often reporting confidence above 85% even when their answer is wrong, and that the problem is worst on hard, knowledge-heavy tasks where overconfidence is not masked by high accuracy. Critically, they observe the same “more reasoning makes it worse” effect from the calibration side: as the reasoning budget grows, a model that is heading towards a wrong answer tends to become *more* confident in it rather than less, effectively building a longer, more elaborate justification for the same mistake. The authors compare this to a human tendency to become increasingly certain of an answer the longer it is considered, even if that answer remains incorrect.

These two studies point to a consistent picture. The doubt is often present *inside* the reasoning trace, in hedges, second-guessing, and self-correction, but it is flattened out by the time the model commits to a final answer, and additional reasoning tends to harden rather than soften that final confidence. The reasoning trace therefore appears to hold uncertainty information that the final answer, and the confidence attached to it, does not reflect.

2.6 Summary and Research Gap

The literature establishes three things relevant to this thesis. First, LLMs, and reasoning models in particular, are deployed in settings where confident errors are costly, yet they are frequently overconfident and poor at recognising when they should abstain. Second, a wide range of UQ methods exists, but the most established ones, semantic entropy, $P(\text{True})$, and verbalised confidence, operate on the final answer (or on the model’s output probabilities), not on the reasoning process. Third, reasoning models produce a rich intermediate trace that has been observed to contain expressions of uncertainty which never reach the final answer, and only a small amount of work (such as CoT-UQ) has begun to use the trace as a source of uncertainty signal.

The gap we address is therefore the following: whether simple, interpretable, black-box features taken directly from a reasoning model’s trace, such as how long it reasons, how much it repeats itself, and how much it hedges, can serve as a usable uncertainty signal for predicting answer correctness, how they compare with the established final-answer baselines, and whether any such effect is specific to reasoning-tuned models or appears whenever a model is prompted to reason step by step. The remainder of this thesis investigates these questions.

3. Methodology

Our methodology consists of three parts. First, the data generation pipeline prompts the models to produce a reasoning trace and a final answer. Second, a post-processing step extracts features from the reasoning trace alongside standard baseline signals. Third, an analysis step builds a logistic regression model from the trace features to compare it against baseline uncertainty quantification (UQ) methods. We use this same experimental setup for both reasoning and non-reasoning models, and across multiple datasets.

The goal is to test whether signals taken from *how* a model reasons (its trace) can tell us whether its answer is likely to be correct, and whether these signals add anything over methods that look only at the final answer. By running the same pipeline on non-reasoning models prompted to reason step by step with chain-of-thought (CoT) (Wei et al., 2022; Kojima et al., 2022), and on more than one dataset, we can also test whether any effect is specific to reasoning-tuned models or holds more generally.

3.1 Data Generation

3.1.1 Models

We use both reasoning and non-reasoning models. This lets us check whether any uncertainty signal in the trace is an effect of the reasoning fine-tuning that reasoning models go through, or whether a non-reasoning model prompted with chain-of-thought exhibits the same behaviour. The models we use are:

- **Qwen3-4B** (Qwen, 2025a): a Qwen 3 series model that can switch between thinking and non-thinking mode by passing a boolean to the `enable_thinking` parameter at generation time. This switch is useful because it lets us compare the same model, with the same weights, in reasoning and non-reasoning mode.
- **DeepSeek-R1-Distill-Llama-8B** (DeepSeek-AI, 2025): a model created by fine-tuning the Llama-3.1-8B base model on samples generated by DeepSeek-R1. It always produces a reasoning trace.
- **QwQ-32B** (Qwen, 2025b): a larger reasoning model trained with reinforcement learning for extended reasoning.
- **Llama-3.1-8B-Instruct** (Meta, 2024): a non-reasoning instruction-tuned model. It is a suitable comparison because it shares its base with DeepSeek-R1-Distill-Llama-8B, so any difference in trace behaviour between the two can be attributed to the reasoning fine-tuning rather than to a different underlying model.

Non-reasoning models do not produce a reasoning trace by default, so we elicit one with a chain-of-thought prompt. We use the zero-shot CoT prompt of Kojima et al. (2022), adapted for the multiple-choice setting:

Answer the following multiple-choice question. Let's think step by step, then give your final answer as a single letter.
[question + options]

Only the essential structure of the prompt is presented here; the complete prompt template used in the experiments is provided in Appendix B. The same CoT prompt is used for all generations (greedy and

samples) of the non-reasoning models.

3.1.2 Datasets

- **MedQA** (Jin et al., 2021): a medical question-and-answer dataset drawn from medical licensing exams. It is a multiple-choice dataset and a good fit for our case: the questions span a range of difficulty and often need multi-step reasoning. Uncertainty and logical gaps matter a lot in this domain because medicine is a high-stakes setting, which makes it a strong primary choice for this project. Because the answers are single choices, correctness is unambiguous to score.
- **MMLU-Pro** (Wang et al., 2024): a broad-domain multiple-choice benchmark covering many subjects, with ten answer options per question rather than the more common four or five. Compared with MedQA, it tests generalisation across domains and uses a larger option set, which also affects the answer-level semantic entropy baseline (see Section 3.2.2).
- **TriviaQA** (Joshi et al., 2017): a free-form (open-answer) question-answering dataset. Unlike the two multiple-choice datasets, the final answer is a short text span rather than an option letter, so it tests whether our findings hold when the answer space is open-ended. This change of format also requires a different form of the semantic entropy baseline, described below.

3.2 Feature Engineering and Extraction

When reasoning models are prompted, they generate intermediate reasoning steps, also called reasoning traces, to break complex problems into multiple steps before giving the final answer. In their output, these reasoning traces are wrapped in `<think>` and `</think>` tags, which makes it straightforward to split the output into a reasoning trace and a final answer. Non-reasoning models prompted with CoT also produce a multi-step explanation before the final answer, but without any separating tag. To split their output we rely on the structure imposed by the CoT prompt, which instructs the model to give a single final answer at the end: we treat the explanation before that final answer as the reasoning trace, and the concluding answer statement as the final answer. This keeps the meaning of the trace/answer split consistent across both model types, even though the delimiter differs.

We extract several properties about each model's answer and reasoning. Some of these are computed during generation and some after generation. This operational split is motivated primarily by computational efficiency: a feature like semantic entropy can be computed after generation from the saved samples, while a feature like verbalised confidence must be requested from the model during generation. Based on where the signal comes from, we group the features into *trace features* (computed from the reasoning trace) and *final-answer features*. The final-answer features are standard, widely used baseline uncertainty signals.

All trace features are computed on the reasoning portion of the output only (the text the model produces before its final answer), and are averaged over the ten sampled generations (described in Section 3.4) unless stated otherwise. Lexicon-based features (hedging and connectors) are normalised per token so that they are not simply a restatement of trace length, which is itself a separate feature.

Because the sampled features require ten generations per question, we also compute a single-pass variant of each text-based feature from the greedy trace alone, which uses one generation. This lets us assess later how much of the signal survives without sampling. The trace-disagreement feature is the exception: it is defined over multiple samples and has no single-trace analogue, so it is absent from the single-pass variant.

3.2.1 Trace Features

- **Trace length:** we count the number of tokens in the reasoning trace using the model’s own tokenizer. The intuition is that if a model is uncertain about its answer, it may reason longer (Kirichenko et al., 2025).
- **Hedging word density:** while reasoning, models tend to use words that signal hesitation, backtracking, and uncertainty. (Kirichenko et al., 2025) observe that the reasoning traces of reasoning models contain more explicit expressions of uncertainty than their final answers, including self-critique markers such as “Wait”, yet the models still produce definitive final answers. They note that exploiting this trace uncertainty is promising but leave it unrealised. This motivates us to use this observation as a concrete and lightweight feature. We use a hedge lexicon developed by (Ulinski and Hirschberg, 2019), which groups hedge terms into *relational* hedges and *propositional* hedges. Relational hedges express the speaker’s relationship to the proposition (in our case, the model’s stance toward its own reasoning, e.g. “I think”, “maybe”, “suppose”), while propositional hedges introduce ambiguity into the content itself (e.g. “approximately”, “roughly”, “about”).

For our purpose of capturing the model’s uncertainty about its own answer, the relational hedges are the more appropriate signal, since propositional hedges can appear even when the model is confident (for example, stating a value “approximately” need not reflect any doubt about the answer). We therefore use the relational hedges as our primary hedging feature, and additionally include hedge terms that are common in reasoning models but absent from the original lexicon, such as “Wait”. We report the propositional hedges separately as a contrast to check whether they carry any signal. All lexicons are frozen before any feature is computed, so that they cannot be tuned to the results.

- **Logical/deductive connector density:** we measure the density of deductive connector words such as “therefore” and “consequently”. We treat this as an exploratory, structural property of the trace and make no prior assumption about its direction; that is, we do not assume more connectors mean more (or less) certainty. We include it to test whether it carries any signal at all.
- **Trace disagreement (divergence):** we measure how much the ten sampled reasoning traces disagree with one another. Each full trace is embedded with the BGE-M3 embedder (Chen et al., 2025), which has a long context window so the whole trace fits without truncation. We then compute the mean pairwise cosine dissimilarity between the ten trace embeddings. For two embedding vectors u and v , the cosine dissimilarity is

$$d(u, v) = 1 - \frac{u \cdot v}{\|u\| \|v\|} \quad (3.2.1)$$

and the feature is the mean of $d(u, v)$ over all pairs of the ten traces. We use an embedding-based measure rather than natural language inference because the traces are long free-form paragraphs that do not fit standard entailment models, and because we want to capture whether the reasoning *paths* diverge, not just whether the final answers differ.

- **Repetition score:** we run a sliding n -gram window over the reasoning trace to measure how repetitive it is. Reasoning models sometimes become stuck in reasoning loops, and circular reasoning may indicate a reasoning failure or logical collapse; repetition is a standard symptom of degenerate generation (Welleck et al., 2019; Holtzman et al., 2020). We compute the fraction of repeated n -grams in each trace (one minus the ratio of distinct n -grams to total n -grams), using $n = 5$ as the primary value and reporting $n = 3$ and $n = 4$ as a robustness check. Before extracting n -grams we convert the trace to lowercase and collapse repeated white-space. This

feature is especially relevant to the abstention use-case, where the question may be unanswerable and the model may loop instead of settling.

3.2.2 Baseline Features

- **Semantic entropy:** a Shannon entropy-based uncertainty quantification technique (Kuhn et al., 2023; Farquhar et al., 2024). Given a question Q , a generative model induces a distribution $P(A | Q)$ over semantically distinct answers, whose Shannon entropy serves as a natural uncertainty measure. Since exact computation is infeasible, we approximate it via Monte-Carlo sampling, clustering semantically equivalent answers, and computing entropy over the resulting clusters. The clustering procedure depends on the answer format:
 - For the multiple-choice datasets (MedQA and MMLU-Pro), the final answers are single option letters, so no clustering is required; semantic entropy reduces directly to the Shannon entropy over the distribution of chosen letters across the ten samples.
 - For the free-form dataset (TriviaQA), the answers are short text spans, so we cluster them by meaning using bidirectional entailment with a natural language inference model (Kuhn et al., 2023), and compute the entropy over the resulting clusters.

Formally, let p_c denote the proportion of samples assigned to cluster c . The cluster entropy, measured in bits, is defined as shown in Equation 2.4.4.

To make semantic entropy comparable with the other confidence signals on the calibration metric, we convert it to a confidence in $[0, 1]$ by normalising against the maximum possible entropy and inverting it,

$$\text{conf} = 1 - \frac{H}{\log_2 K} \quad (3.2.2)$$

where K is the number of answer options ($K = 5$ for MedQA and $K = 10$ for MMLU-Pro). This normalised-entropy confidence is a standard construction for placing entropy on a comparable $[0, 1]$ scale across tasks with different numbers of options. A high entropy (samples disagree) maps to low confidence, and a low entropy (samples agree) maps to high confidence.

- **P(True):** a self-evaluation UQ method (Kadavath et al., 2022). After the model produces an answer, we present the question together with its proposed answer and ask the model to judge whether the answer is correct, choosing between “True” and “False”. We then read the model’s probability for the “True” token and normalise it against the “False” token as shown in the Equation 2.4.5.

A higher value means the model judges its own answer more likely to be correct.

- **Verbalised confidence:** a simple and widely used signal in which the model is asked to state its confidence directly (Tian et al., 2023; Lin et al., 2022). We ask the model to state how confident it is in its answer on a numeric scale from 0 to 100, and rescale this to a $[0, 1]$ value.

3.3 Analysis

After extracting the features, we build a logistic regression model that uses the trace features to predict whether the final answer is correct or incorrect. We compare the trained model against the baseline methods using the following metrics.

- **AUROC**: the area under the receiver operating characteristic curve. It is a standard metric in the UQ literature (Kuhn et al., 2023; Farquhar et al., 2024; Zhang and Zhang, 2025; Tian et al., 2023) and measures the probability that a method ranks a randomly chosen correct answer above a randomly chosen incorrect one. It is threshold-free, which makes it a clean measure of how well an uncertainty signal separates correct from incorrect answers, and it matters for downstream tasks such as hallucination detection and abstention.
- **Risk-coverage**: a selective-prediction metric. If a method answers only the fraction of questions it is most confident about (the coverage) and abstains on the rest, the risk-coverage curve shows the accuracy on the answered subset as coverage varies. We summarise it with the area under the risk-coverage curve and with the accuracy at a fixed 80% coverage. This measures how useful an uncertainty signal is for deciding when to trust the model, which is the practical setting for abstention.
- **Expected Calibration Error (ECE)**: a widely used metric (Mei et al., 2025; Tian et al., 2023) for whether a confidence signal is calibrated, that is, whether a stated confidence of, say, 70% corresponds to being correct about 70% of the time. We bin predictions by their confidence into M equal-width bins and take the weighted average of the gap between average confidence and accuracy in each bin as shown in the Equation 3.3.1:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)| \quad (3.3.1)$$

where B_m is the set of predictions in bin m , $|B_m|$ its size, and N the total number of predictions. We compute ECE only for methods that output a probability (the logistic regression models, the normalised semantic entropy confidence defined above, $P(\text{True})$, and verbalised confidence), since the raw trace features are not probabilities.

In addition to the expected calibration error, we report two proper scoring rules, the Brier score (Equation 3.3.2) and the negative log-likelihood (Equation 3.3.3), which jointly reward calibration and sharpness. For predicted probabilities $\hat{p}_n$ and binary outcomes y_n over N questions,

$$\text{Brier} = \frac{1}{N} \sum_{n=1}^N (\hat{p}_n - y_n)^2 \quad (3.3.2)$$

$$\text{NLL} = -\frac{1}{N} \sum_{n=1}^N [y_n \log \hat{p}_n + (1 - y_n) \log(1 - \hat{p}_n)] \quad (3.3.3)$$

these probability-based scores we place each baseline on an equal footing with the fitted model by mapping its raw score to a probability with Platt scaling, a one-parameter logistic fit, estimated inside the training folds only.

Alongside these comparison metrics, we use three descriptive analyses to characterise the individual trace features. These are applied to a feature set that is fixed in advance (Section 3.4.2); they describe the features rather than select them.

- **Single-feature AUROC**: For each feature we compute the AUROC obtained when that feature is used on its own as the score, which measures how well it separates correct from incorrect answers in isolation.
- **Cohen's d** : To describe the direction and magnitude of a feature's separation between correct and incorrect answers, we use the standardised mean difference. For a feature with means μ_c and

μ_i on the correct and incorrect groups and pooled standard deviation s_p ,

$$d = \frac{\mu_c - \mu_i}{s_p}, \quad s_p = \sqrt{\frac{(n_c - 1)s_c^2 + (n_i - 1)s_i^2}{n_c + n_i - 2}} \quad (3.3.4)$$

where n_c, n_i and s_c, s_i are the group sizes and standard deviations. A positive d indicates a feature that is higher on correct answers, a negative d one that is higher on incorrect answers.

- **Leave-one-feature-out (LOFO):** To describe how much each feature contributes once the others are present, we remove one feature at a time from the full model, refit under the same cross-validation procedure, and record the resulting change in AUROC. A large drop indicates a feature that contributes signal not already captured by the others; a change near zero indicates redundancy with the remaining features.

3.4 Experimental Setup

3.4.1 Data Generation

For each dataset we subsampled 1000 questions using a fixed random seed. Fixing the seed serves two purposes: it makes the sample reproducible, so that the same 1000 questions are drawn on any re-run of the pipeline, and it ensures that every model and condition in our study is evaluated on the identical question set, so that differences between models reflect the models rather than differences in the sampled questions.

All generation used the vLLM inference engine on Google Cloud “a2-highgpu” instances. The four sub-32B models (Qwen3-4B, DeepSeek-R1-Distill-Llama-8B, Qwen3-4B with thinking disabled, and Llama-3.1-8B-Instruct) ran on a single A100-40GB (a2-highgpu-1g, 85GB RAM, 200GB disk). The 32B model (QwQ-32B) does not fit on a single 40GB device, so for that model the instance was resized to a2-highgpu-2g (two A100-40GBs connected by NVLink, 170GB RAM, 300GB disk), and vLLM was launched with tensor-parallel size 2. Further details are in the Appendix A.

- **Greedy answer (temperature = 0):** we prompt the model to reason and give its chosen option. We use this greedy answer to measure accuracy on the final answer, and as the answer that the P(True) and verbalised-confidence passes judge.
- **Verbalised confidence pass:** we present the question, the reasoning trace, and the greedy final answer, and prompt the model to state its confidence in that answer. We use a separate pass rather than asking for confidence in the same generation as the answer because asking for confidence inline can change the answer itself, the model may reason differently when it knows it must also produce a confidence score. A separate pass keeps the original answer untouched and elicits the confidence judgment cleanly.
- **P(True) pass:** we present the question, the reasoning trace, and the greedy final answer, and ask the model to judge whether the answer is correct. For both the verbalised-confidence and P(True) passes we allow the model to reason before giving its judgment, rather than forcing an immediate answer. There are two reasons. First, the reasoning models are trained to always produce a reasoning block before any output, so reading the first generated token would capture a reasoning token rather than the judgment; for the non-reasoning models the CoT prompt similarly elicits reasoning before the verdict, so in both cases the verdict is read at the end. Second, forcing an immediate answer would put the reasoning models in a mode they were not trained for, which

would itself bias the measurement. Letting the model reason and reading the verdict at the end is therefore the more faithful, and more consistent, measurement across model types.

- **Sampled generations (temperature = 0.7, $n = 10$):** we prompt the model ten times at high temperature. These samples let us compute the features that need a distribution over outputs: the Shannon entropy over the final answers (semantic entropy), the trace disagreement, and the trace features averaged over samples (mean trace length, hedging density, connector density, and repetition score).

All generated outputs are parsed by a parser that separates the reasoning trace from the final answer and extracts the chosen option. For the multiple-choice datasets (MedQA and MMLU-Pro), the semantic entropy baseline is computed over the distribution of chosen option letters across the sampled answers, since on a multiple-choice question the option label already defines the answer's meaning and no further clustering is needed. For the free-form dataset (TriviaQA), where the same answer can be written in many different surface forms, we instead cluster the sampled answers by meaning before computing entropy: two answers are placed in the same cluster when they bidirectionally entail each other, following the semantic-clustering approach of Kuhn et al. (Kuhn et al., 2023). We perform this entailment check using a DeBERTa-v3-large model fine-tuned on MNLI (He et al., 2021). After generation is complete, we compute the remaining post-processing features. The trace embeddings for the disagreement feature are produced with the BGE-M3 embedder (Chen et al., 2025), using the mean pairwise cosine dissimilarity defined above.

3.4.2 Feature Selection and Modelling

Before modelling, we check for redundant features by examining the pairwise correlations between trace features. Where two features are almost perfectly correlated and therefore carry the same information (for example, the 3-, 4-, and 5-gram repetition variants), we keep a single representative rather than all of them. This avoids introducing the same predictive signal multiple times into the model.

To evaluate feature utility, perform predictive modelling, and compare against baselines, we execute our analysis through the following structured steps:

- **Feature Characterisation:** We use two complementary checks to understand which features carry a useful signal. First, we compute the single-feature AUROC to measure how well each feature separates correct from incorrect answers in isolation. Second, we apply a leave-one-feature-out (LOFO) procedure by dropping each feature in turn, refitting the model, and measuring the performance change to determine how much unique signal it adds on top of the others. We also report Cohen's d , a standardised effect size, to describe the direction and size of each feature's separation between the two groups.
- **Model Training and Validation:** We train a logistic regression model to predict whether the final answer is correct using the uncertainty features as input. We implement stratified five-fold cross-validation, meaning the data is split into five parts while keeping the proportion of correct and incorrect answers identical across folds. All fitting and feature standardisation are performed strictly inside the training folds to ensure no information leaks from the held-out test data. Confidence intervals for the AUROC are obtained via bootstrap resampling of the held-out predictions. A separate logistic regression classifier is trained and evaluated independently for each (model, dataset) cell. We do not pool data across models or datasets, and we make no assumption that a single classifier generalises across model architectures or task domains.
- **Data Filtering:** We run the comparison exclusively on a clean subset of the data containing

questions for which no generation was truncated at the token limit. This step ensures that structural trace-length and repetition features are not distorted by text that was cut off mid-generation. While this filtering choice could potentially omit exceptionally complex or lengthy responses, it is methodologically necessary to protect the accuracy of the trace features.

- **Baseline Comparison and Evaluation:** Finally, we compare our trace-based logistic regression model against the baseline UQ methods across three core dimensions: discrimination (evaluated using AUROC), selective prediction (evaluated using the risk-coverage curve and the area under it), and calibration (evaluated using Expected Calibration Error).
- **Calibration Rules and Controls:** Because Expected Calibration Error (ECE) can be misleadingly lowered if a method simply hedges its predictions toward the baseline accuracy rate, we additionally report proper scoring rules: the Brier score and the negative log-likelihood. To guarantee a fair comparison, we calibrate each baseline score with a simple one-parameter mapping (Platt scaling) fitted inside the cross-validation folds, allowing us to compare the quality of the underlying signals rather than the quality of model fitting. The identical evaluation pipeline is also run on non-reasoning models prompted with chain-of-thought to test if our findings are specific to reasoning-tuned models or hold more generally.

4. Results

4.1 Data and Exploratory Analysis

4.1.1 Generation and Clean Data

Generations were produced for every model and dataset combination, with the exception of QwQ-32B on MedQA, which was not run due to time and cost constraints. Most cells exhibit a tolerable truncation rate. Truncation is more common for the reasoning models, and is highest on MMLU-Pro, the dataset on which these models also produce their longest reasoning traces. The two are linked, since a longer trace is more likely to reach the token limit. The R1-Distill-Llama-8B model represents the lowest-performing configuration on MMLU-Pro, combining the lowest accuracy with the highest truncation rate, where 24.3 per cent of generations were truncated and only 61.4 per cent of questions remained clean. As expected, the non-reasoning models produce much shorter traces and truncate far less often. Because a question is retained only when neither its greedy generation nor any of its ten samples was truncated, the clean fraction is lower than the greedy-only truncation rate alone would suggest.

Table 4.1: Generation statistics per (model, dataset): the percentage of questions retained as clean and labelled, greedy accuracy on that clean set, mean sampled trace length in tokens, and the greedy truncation rate. A question is clean only if neither its greedy generation nor any of its ten samples was truncated, so the clean percentage is lower than the greedy truncation rate alone implies.

Dataset	Model	Clean (%)	Accuracy	Mean Trace Length	Truncated (Greedy)(%)
MedQA	Qwen3-4B	74.0	0.764	1754.0	10.1
MedQA	R1-Distill-Llama-8B	79.6	0.522	1595.2	7.5
MedQA	Qwen3-4B-NoThink	92.6	0.642	623.3	3.4
MedQA	Llama-3.1-8B-Instruct	97.6	0.651	416.0	1.6
MMLU-Pro	Qwen3-4B	73.0	0.751	2468.1	18.7
MMLU-Pro	R1-Distill-Llama-8B	61.4	0.496	2329.9	24.3
MMLU-Pro	Qwen3-4B-NoThink	91.8	0.659	678.8	5.8
MMLU-Pro	Llama-3.1-8B-Instruct	73.3	0.538	782.2	15.2
MMLU-Pro	QwQ-32B	65.7	0.90	2828.3	21.2
TriviaQA	Qwen3-4B	91.5	0.492	679.4	6.5
TriviaQA	R1-Distill-Llama-8B	94.4	0.381	557.0	2.4
TriviaQA	Qwen3-4B-NoThink	98.9	0.419	69.5	0.2
TriviaQA	Llama-3.1-8B-Instruct	81.8	0.626	273.1	9.2
TriviaQA	QwQ-32B	95.6	0.714	821.8	2.3

In this sample, an observed pattern is that the reasoning models produce substantially longer traces than the non-reasoning models across all datasets, averaging roughly 1,629 tokens against 474 tokens for the non-reasoning controls. This is the very material from which our trace features are computed, and the wide range in trace length, from around 70 tokens on the shortest non-reasoning cell to nearly 2,850 on the longest reasoning cell, is what makes trace length a plausible discriminative signal in the modelling that follows.

4.1.2 Hedging Vocabulary in Reasoning Traces

Analysing the traces of all reasoning models, we find that the most frequent hedging terms are reasoning-specific markers such as “wait” and “maybe”, rather than the formal hedges in the base lexicon. These were among the terms we added to the lexicon precisely because reasoning-tuned models are trained

to self-correct, and such markers are characteristic of that behaviour. Their prominence is consistent with Muennighoff et al. (2025), who show that forcing a model to continue reasoning by appending “Wait” to its trace increases test-time computation and can improve accuracy.

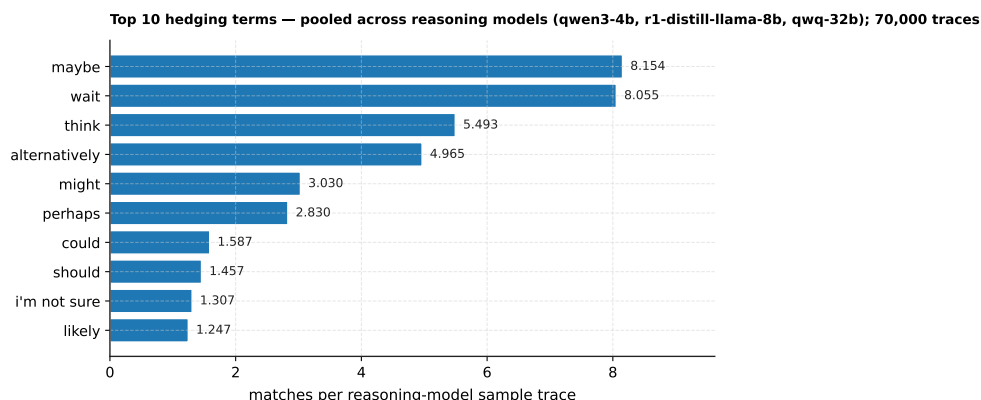


Figure 4.1: The ten most frequent hedging terms by per-trace frequency, pooled across the sampled traces of the three reasoning models. Reasoning-specific markers such as “wait” and “maybe” dominate over the formal hedges in the base lexicon.

4.2 Feature Analysis

We now characterise the individual trace features: how they relate to one another, how well each predicts correctness on its own, the direction and size of their separation between correct and incorrect answers, and how much each contributes once the others are present. This analysis is descriptive; the feature set used for the classifier is fixed in Section 4.3.

4.2.1 Feature Correlation

The three repetition variants are almost perfectly correlated (all $|r| > 0.95$), so we retain only the 5-gram variant (rep-5) and drop the 3- and 4-gram versions. Formal and combined hedging are similarly redundant, so the combined variant is dropped in favour of the formal and reasoning split. Trace length and rep-5 are correlated but well below the redundancy threshold, and are therefore both retained. Connector density shows only weak correlation with the other features.

4.2.2 Single-Feature Discrimination

Considered individually, rep-5 and trace length are the strongest predictors of correctness on the multiple-choice datasets, followed by formal hedging. Connector density is the weakest single predictor throughout. Trace divergence is weak on the multiple-choice datasets but is a strong single predictor on the free-form dataset (TriviaQA), where its AUROC reaches the high 0.70s across the reasoning models. The formal and reasoning hedging signals track each other closely on the non-reasoning models, but separate on the reasoning models, particularly on free-form questions, where reasoning hedging becomes the stronger of the two. Table 4.2 reports the single-feature AUROC for the Qwen3-4B / MedQA cell as a representative example; the full per-cell values are given in Appendix C.

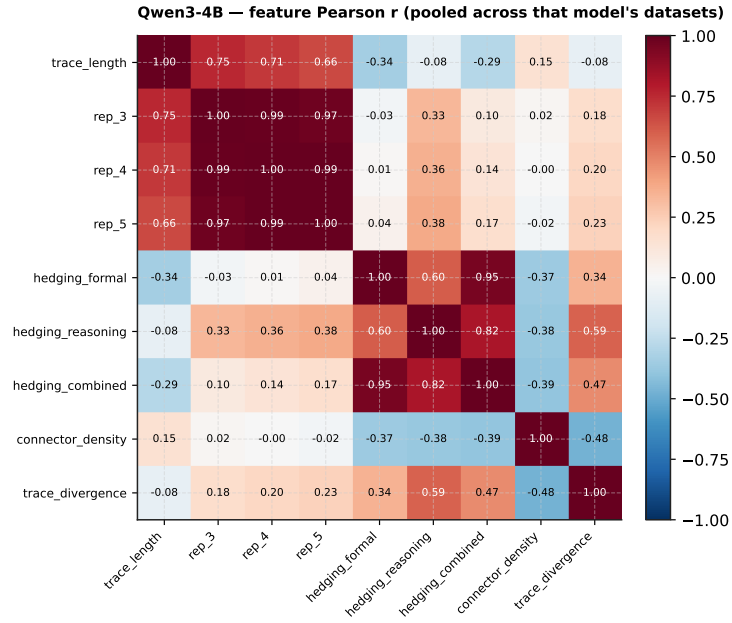


Figure 4.2: Correlation matrix of the trace features for Qwen3-4B.

Table 4.2: Single-feature AUROC on the Qwen3-4B / MedQA cell.

Feature	n	AUROC	95% CI	Strength
Trace Length	740	0.750	[0.713, 0.788]	0.250
Rep-5	740	0.755	[0.718, 0.790]	0.255
Hedging (Formal)	740	0.684	[0.638, 0.725]	0.184
Hedging (Reasoning)	740	0.628	[0.581, 0.672]	0.128
Connector Density	740	0.546	[0.505, 0.596]	0.046
Trace Divergence	740	0.519	[0.501, 0.569]	0.019

4.2.3 Effect Sizes

Cohen's d confirms the direction of these signals. On the reasoning models, trace length, repetition, formal and reasoning hedging, and trace divergence are all higher on incorrect answers (negative d), while connector density is higher on correct answers. Trace length and rep-5 show the largest negative effects on the reasoning models. Trace divergence again shows the task dependence, a small effect on the multiple-choice datasets but a substantially larger one on TriviaQA across all reasoning models. On the non-reasoning models the effects are present but weaker.

4.2.4 Feature Distributions by Correctness

The distributions split by correctness show the same pattern. On the multiple-choice datasets, trace length and rep-5 separate the correct and incorrect answers most clearly, while connector density overlaps heavily. On TriviaQA, trace divergence separates the two classes, with lower divergence on the correct answers.

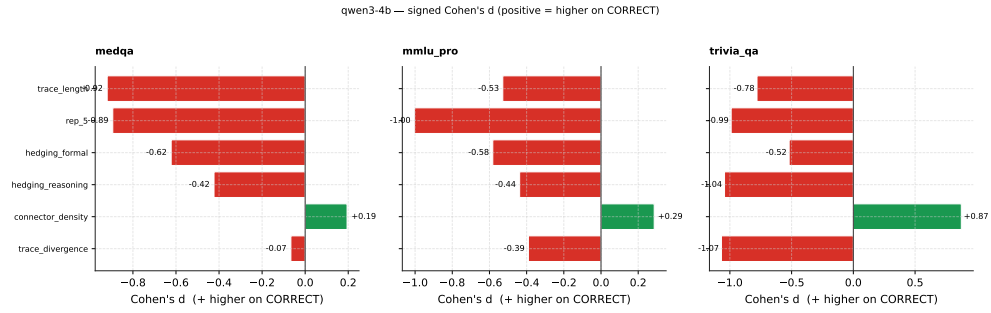
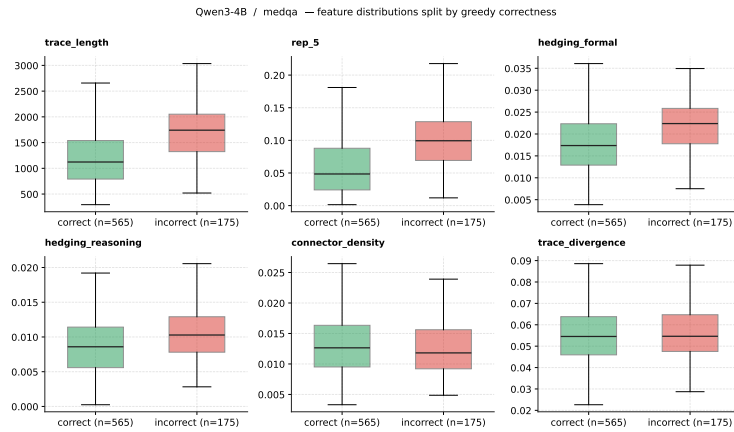
Figure 4.3: Signed Cohen's d of each feature on correctness for Qwen3-4B, by dataset.

Figure 4.4: Feature distributions split by answer correctness for Qwen3-4B / MedQA.

4.2.5 Feature Contribution

The leave-one-feature-out analysis shows how much each feature contributes once the others are present, which differs from its strength in isolation. On the reasoning multiple-choice cells, rep-5 and formal hedging are the largest contributors. Trace length, although a strong single predictor, adds almost nothing on top of rep-5, reflecting the correlation between the two: rep-5 captures most of the shared signal. Trace divergence is again task dependent, contributing little on the multiple-choice datasets but materially on TriviaQA. Connector density contributes very little anywhere, with a median change near zero, and is mildly negative on a small number of non-reasoning cells.

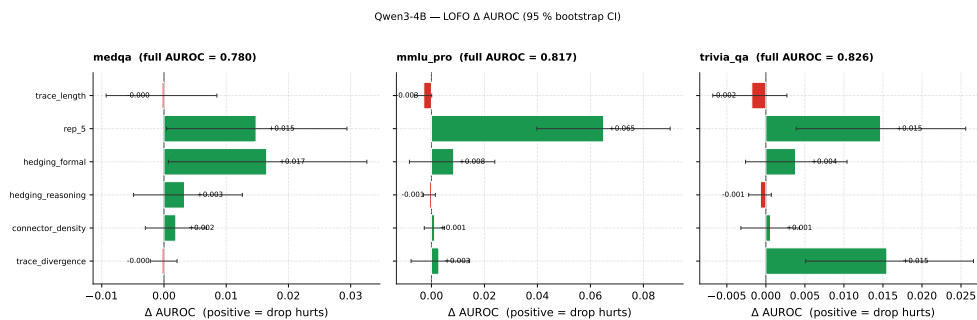


Figure 4.5: Leave-one-feature-out change in AUROC for Qwen3-4B, by dataset.

4.3 The trace_LR Feature Set

The preceding analysis fixes the feature set used by the classifier for the remainder of this chapter. We refer to this model as *trace_LR*: a logistic regression over five trace features, applied identically on every model and dataset cell, namely trace length, the 5-gram repetition score (rep-5), formal hedging, connector density, and trace divergence.

The set was fixed in advance by the hypothesis and reduced only on grounds of redundancy, rather than by ranking features on their evaluation performance. From the six candidate features, we removed the near-duplicates: the three repetition variants are almost perfectly correlated ($|r| > 0.95$), so we kept rep-5 alone, and combined hedging duplicates formal hedging, so it too was dropped. Reasoning hedging was removed because its leave-one-out contribution is negligible once the other features are present. Trace length and connector density are retained despite being, respectively, redundant with rep-5 and weak overall: both were named in the hypothesis, neither is a near-duplicate of another feature, and dropping them after observing their performance would amount to test-set-driven selection, which we avoid. Their contributions are reported honestly in the analysis above. The resulting set is summarised in Table 4.3.

4.4 Comparison with Baseline Methods

We now compare trace_LR against the three baseline methods. Across all cells, semantic entropy is the strongest baseline; the two self-report baselines, P(True) and verbalised confidence, are weaker throughout, and trace_LR exceeds both on all fourteen cells. The substantive comparison is therefore between trace_LR and semantic entropy, reported in Table 4.4 and tested cell by cell in Table 4.5.

The pattern is consistent. Recall that each trace_LR is trained per cell, so these are in-distribution comparisons rather than a claim about a single classifier generalising across cells. On the non-reasoning

Table 4.3: The frozen trace_LR feature set and the excluded candidates.

Feature	Status	Reason
Trace length	Retained	Hypothesis-led; correlated with rep-5 but not redundant
Rep-5	Retained	Primary structural signal
Hedging (formal)	Retained	Strongest hedging contributor
Connector density	Retained	Hypothesis-led; weak but not dropped on performance
Trace divergence	Retained	Task-dependent; contributes on free-form
Rep-3, Rep-4	Excluded	Near-duplicates of rep-5 ($ r > 0.95$)
Hedging (combined)	Excluded	Near-duplicate of formal hedging
Hedging (reasoning)	Excluded	Adds ≈ 0 once other features present (LOFO)

models, semantic entropy achieves the higher AUROC on every dataset, often by a substantial margin. On the reasoning models, the outcome depends on both the model and the task format. The two most capable reasoning models, Qwen3-4B and QwQ-32B, are outperformed by trace_LR on the multiple-choice datasets: trace_LR leads by approximately 0.09 AUROC on both Qwen3-4B cells and by 0.15 on QwQ-32B / MMLU-Pro, the largest margin in the study, with the paired bootstrap favouring trace_LR in 100% of resamples for all three. On the free-form dataset (TriviaQA), by contrast, semantic entropy achieves the higher AUROC for every model, including the reasoning ones. R1-Distill-Llama-8B is the exception among the reasoning models: semantic entropy is stronger on all three of its datasets, although the multiple-choice margins are small and their confidence intervals straddle zero, placing those two cells closer to ties than to clear separations.

In summary, trace_LR exceeds semantic entropy on three of the fourteen cells, all of which are capable reasoning models on multiple-choice tasks, while semantic entropy is the stronger method on free-form questions and on the non-reasoning models throughout.

The remaining discrimination measures broadly agree with this picture. In the three cells where trace_LR has the higher AUROC, it also achieves the lower area under the risk–coverage curve, indicating better selective prediction; accuracy at 80% coverage is closer between the two methods, with semantic entropy marginally ahead on the two Qwen3-4B cells and trace_LR ahead on QwQ-32B / MMLU-Pro. Full values are given in Appendix C.

4.5 Greedy versus Sampled Trace Features

Table 4.4: Comparison of confidence estimation methods. Best AUROC within each model/dataset pair is shown in bold.

Dataset	Model	trace_LR	Semantic Entropy	P(True)	Verbalised Conf.
<i>Reasoning models</i>					
MedQA	Qwen3-4B	0.7767	0.6828	0.5950	0.6407
MMLU-Pro	Qwen3-4B	0.8175	0.7225	0.6022	0.6954
TriviaQA	Qwen3-4B	0.8266	0.8714	0.7799	0.7429
MedQA	R1-Distill-Llama-8B	0.6701	0.6838	0.5851	0.5644
MMLU-Pro	R1-Distill-Llama-8B	0.6977	0.7277	0.6290	0.5614
TriviaQA	R1-Distill-Llama-8B	0.7334	0.8341	0.6865	0.6556
MMLU-Pro	QwQ-32B	0.7436	0.5957	0.6090	0.6289
TriviaQA	QwQ-32B	0.7587	0.8074	0.7150	0.7153
<i>Non-reasoning models</i>					
MedQA	Qwen3-4B-NoThink	0.6528	0.6959	0.6207	0.5925
MMLU-Pro	Qwen3-4B-NoThink	0.6909	0.7613	0.6172	0.6263
TriviaQA	Qwen3-4B-NoThink	0.7964	0.8432	0.7148	0.6155
MedQA	Llama-3.1-8B-Instruct	0.6541	0.7843	0.5501	0.5555
MMLU-Pro	Llama-3.1-8B-Instruct	0.6562	0.7783	0.5901	0.6048
TriviaQA	Llama-3.1-8B-Instruct	0.6478	0.7913	0.6160	0.5625

Table 4.5: Bootstrap comparison of trace_LR against Semantic Entropy. Positive Δ AUROC indicates trace_LR outperforms Semantic Entropy, while negative values indicate the opposite. Confidence intervals are percentile bootstrap intervals.

Dataset	Model	Median Δ AUROC	95% CI	trace_LR Wins (%)
<i>Reasoning models</i>				
MedQA	Qwen3-4B	0.0935	[0.0511, 0.1334]	100.0
MMLU-Pro	Qwen3-4B	0.0969	[0.0518, 0.1343]	100.0
TriviaQA	Qwen3-4B	-0.0442	[-0.0652, -0.0234]	0.0
MedQA	R1-Distill-Llama-8B	-0.0140	[-0.0519, 0.0269]	24.8
MMLU-Pro	R1-Distill-Llama-8B	-0.0321	[-0.0762, 0.0136]	8.0
TriviaQA	R1-Distill-Llama-8B	-0.1004	[-0.1279, -0.0751]	0.0
MMLU-Pro	QwQ-32B	0.1467	[0.0845, 0.2156]	100.0
TriviaQA	QwQ-32B	-0.0486	[-0.0797, -0.0175]	0.0
<i>Non-reasoning models</i>				
MedQA	Qwen3-4B-NoThink	-0.0428	[-0.0828, -0.0044]	1.7
MMLU-Pro	Qwen3-4B-NoThink	-0.0698	[-0.1075, -0.0321]	0.0
TriviaQA	Qwen3-4B-NoThink	-0.0461	[-0.0684, -0.0243]	0.0
MedQA	Llama-3.1-8B-Instruct	-0.1305	[-0.1679, -0.0943]	0.0
MMLU-Pro	Llama-3.1-8B-Instruct	-0.1208	[-0.1664, -0.0771]	0.0
TriviaQA	Llama-3.1-8B-Instruct	-0.1437	[-0.1808, -0.1057]	0.0

We compare the sampled trace_LR against the single-pass variant, trace_LR_greedy, which uses the four features computable from one greedy trace (trace length, rep-5, formal hedging, and connector density); trace divergence is omitted, as it has no single-trace analogue. Table 4.6 reports the difference.

The sampled model achieves the higher AUROC on every cell. The loss from using a single trace is smaller on the multiple-choice datasets (median 0.045 AUROC) than on the free-form datasets (median 0.068), though part of the free-form gap reflects the absent divergence feature rather than sampling alone. On the two cells where trace_LR exceeds semantic entropy, the single-pass model loses only 0.012 AUROC on Qwen3-4B / MMLU-Pro (its interval includes zero) and 0.044 on Qwen3-4B / MedQA; in both cases it remains above all baselines.

Table 4.6: Bootstrap comparison of sampled versus greedy confidence estimation. Positive Δ AUROC indicates sampled outperforms greedy. Confidence intervals are percentile bootstrap intervals.

Dataset	Model	n	Median Δ AUROC	95% CI	Sampled Wins (%)
<i>Reasoning models</i>					
MedQA	Qwen3-4B	740	0.0434	[0.0118, 0.0724]	99.9
MMLU-Pro	Qwen3-4B	730	0.0118	[-0.0110, 0.0356]	83.6
TriviaQA	Qwen3-4B	915	0.0493	[0.0257, 0.0736]	100.0
MedQA	R1-Distill-Llama-8B	792	0.0343	[-0.0032, 0.0717]	96.1
MMLU-Pro	R1-Distill-Llama-8B	599	0.0430	[-0.0001, 0.0864]	97.4
TriviaQA	R1-Distill-Llama-8B	944	0.0642	[0.0273, 0.1010]	99.9
TriviaQA	QwQ-32B	956	0.0679	[0.0374, 0.1011]	100.0
<i>Non-reasoning models</i>					
MedQA	Qwen3-4B-NoThink	926	0.0449	[0.0075, 0.0823]	98.6
MMLU-Pro	Qwen3-4B-NoThink	908	0.0608	[0.0252, 0.0952]	100.0
TriviaQA	Qwen3-4B-NoThink	989	0.1665	[0.1327, 0.2008]	100.0
MedQA	Llama-3.1-8B-Instruct	974	0.0791	[0.0375, 0.1185]	100.0
MMLU-Pro	Llama-3.1-8B-Instruct	731	0.0619	[0.0213, 0.1030]	99.7
TriviaQA	Llama-3.1-8B-Instruct	818	0.1159	[0.0583, 0.1708]	100.0

4.6 Combining Trace Features with Semantic Entropy

We also examine whether combining the trace features with the strongest baseline yields further improvement, by fitting a logistic regression over the five trace features together with semantic entropy. This combined model, full_LR, has access to both sources of signal by construction and is therefore best-or-tied-best on almost every cell; we report it as an upper-bound reference rather than as a proposed method. On the cells where trace_LR already exceeds semantic entropy, adding semantic entropy yields only a small improvement over trace_LR alone (approximately 0.015 AUROC on both Qwen3-4B cells), indicating that the trace features already capture most of the available signal there. On the cells where semantic entropy is the stronger method, the combined model's gain over trace_LR comes mostly from the semantic-entropy component. The two sources are thus largely complementary on the reasoning models and harder tasks, and redundant on the non-reasoning controls.

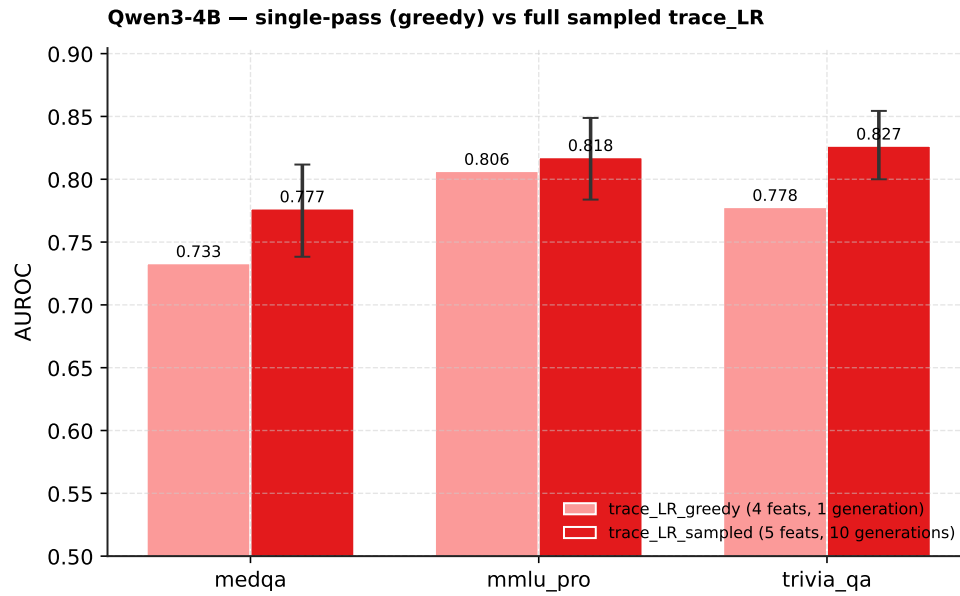


Figure 4.6: Greedy versus sampled trace_LR AUROC for Qwen3-4B across its three datasets

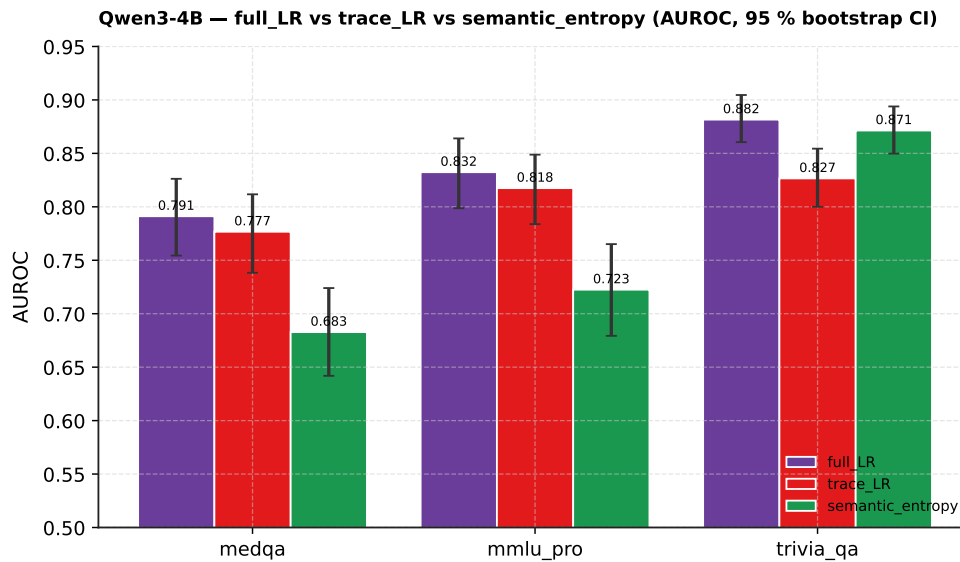


Figure 4.7: AUROC of full_LR, trace_LR, and semantic entropy for Qwen3-4B across its datasets.

4.7 Calibration

The baseline methods include the models' own expressed confidence, and we first note that these self-reported signals are poorly calibrated. As Table 4.7 shows, the expected calibration error of $P(\text{True})$ and verbalised confidence is high, and across the full set of cells both are systematically overconfident, assigning high confidence to answers that are often incorrect. Semantic entropy, treated as a confidence score, is better calibrated than the self-reports but is still some distance from being well calibrated. This miscalibration of the models' own confidence is part of the motivation for estimating uncertainty externally rather than relying on what the model reports.

Table 4.7: Expected calibration error (ECE; lower is better) for different confidence estimation methods.

Dataset	Model	$P(\text{True})$	Verbalised Confidence	Semantic Entropy
MedQA	Qwen3-4B	0.2347	0.2005	0.1626
MMLU-Pro	Qwen3-4B	0.3024	0.2373	0.1922
TriviaQA	Qwen3-4B	0.4379	0.4421	0.2166

We do not, however, use calibration error to rank the methods. Expected calibration error can be made small by predictions that are uninformative: a method that simply reports a value close to the overall accuracy for every question attains a near-zero calibration error while carrying no discriminative signal. For this reason we assess discrimination as the primary criterion, and treat calibration error only as evidence that the raw self-reports are untrustworthy. A fuller calibration analysis, including proper scoring rules that are not subject to this artefact, is provided in Appendix D.

4.8 Discussion

The results indicate that reasoning traces contain useful information for uncertainty estimation beyond the model's reported confidence. Among the proposed trace-based features, trace length and repetition consistently exhibited the strongest predictive signal across datasets and models. One possible explanation is that both features reflect the amount of cognitive effort the model expends when constructing an answer. Longer traces and increased repetition may emerge when the model encounters uncertainty or explores multiple solution paths before reaching a final response. This observation is consistent with the view that uncertainty is manifested not only in the final prediction but also throughout the reasoning process itself.

In contrast, connector-word usage showed a weaker individual effect despite being positively associated with correctness. This suggests that discourse structure alone is insufficient for reliably identifying uncertainty. While connector words may capture aspects of reasoning coherence and logical progression, they appear to provide only complementary information when compared with stronger behavioural indicators such as trace length and repetition. Their contribution may therefore be most valuable when combined with other trace features rather than when used in isolation.

The behaviour of trace divergence further highlights the importance of task characteristics. The feature demonstrated a stronger signal on free-form generation datasets than on multiple-choice benchmarks. A plausible explanation is that free-form tasks allow substantially greater variation in reasoning trajectories, making divergence between traces more informative. In multiple-choice settings, the constrained output space may reduce the diversity of reasoning paths and consequently limit the usefulness of divergence-based measurements. This finding suggests that some trace features are inherently task-dependent and should be selected with consideration of the target application.

Another notable observation is the reduced effectiveness of trace features on non-reasoning models. Models that do not explicitly generate extended reasoning traces provide less information from which to extract behavioural uncertainty signals. As a result, the relationship between trace characteristics and prediction correctness becomes weaker. This supports the broader interpretation that the value of trace-based uncertainty estimation depends on the presence of meaningful intermediate reasoning rather than merely the generation of additional text.

The comparative performance across benchmark methods also provides insight into the conditions under which trace-based approaches are most effective. The strong performance of the proposed method on multiple-choice datasets may be related to the limitations of semantic entropy in constrained answer spaces. When the set of possible outputs is small, semantic variation between responses is naturally reduced, causing semantic entropy to approach lexical entropy and potentially diminishing its informativeness. In such settings, reasoning-trace characteristics may provide a richer source of uncertainty information than output diversity alone.

An important practical finding is that useful uncertainty signals remain available even when features are extracted from a single greedy-generated trace. Although sampled traces generally provide additional information, the greedy setting substantially reduces computational requirements by eliminating the need for repeated generation. This suggests that trace-based uncertainty estimation can remain effective under realistic deployment constraints where inference cost and latency are important considerations.

Overall, these findings support the interpretation that uncertainty is reflected in observable properties of the reasoning process itself. Rather than relying exclusively on output probabilities or response diversity, examining the structure and behaviour of reasoning traces provides an alternative perspective on model uncertainty that complements existing approaches.

5. Conclusion and Future Work

This work investigated whether reasoning traces can be used as a source of uncertainty information in both reasoning and non-reasoning language models employing Chain-of-Thought (CoT) prompting. The experimental results show that features extracted from reasoning traces contain meaningful uncertainty signals and, in several settings, provide better predictive performance than existing uncertainty estimation baselines. The findings also show that these signals are particularly informative for reasoning-oriented models, where the generated traces provide useful information for prediction.

The proposed features are simple to compute and require only information available in the generated reasoning trace, allowing them to be combined within a lightweight classifier. Across the evaluated datasets, trace-based uncertainty estimation remained effective even when the features were extracted from a single greedy generation. This reduces the need for multiple sampled generations and makes the approach more practical for deployment scenarios.

The analysis further shows that different trace properties contribute differently to uncertainty estimation. Features related to trace length and repetition consistently exhibited strong predictive value, while other features showed dataset-dependent behaviour. These results indicate that reasoning traces contain multiple complementary indicators of uncertainty that can be exploited without access to model internals or additional model training.

These findings support the use of lightweight trace-based classifiers as an alternative or complementary approach to existing uncertainty estimation methods, particularly in settings where computational efficiency is important.

5.1 Limitations

Several limitations should be considered when interpreting the results of this study. The experiments were conducted on a limited set of reasoning models, and it remains unclear whether the observed feature importance and classifier performance generalise across different reasoning architectures and model scales. A related constraint is that we train a separate classifier for each model and dataset by design, and we did not test whether a classifier trained on one cell transfers to another. Cross-model and cross-domain transfer of trace_LR therefore remains an open question and a natural direction for future work. The evaluation was also restricted to a small number of datasets and task types, and the results indicate that the effectiveness of trace-based uncertainty signals can vary depending on task characteristics. Furthermore, the feature set explored in this work was intentionally limited to simple and easily obtainable trace properties; more sophisticated linguistic, structural, or semantic features may provide additional uncertainty information. The observed relationships between trace characteristics and uncertainty are also predictive rather than causal, and recent work questioning the faithfulness of reasoning traces suggests that the reliability of trace-based uncertainty estimation may depend on how accurately generated traces reflect the model’s underlying reasoning process.

5.2 Future Work

Future work can extend this study by evaluating trace-based uncertainty estimation across a wider range of reasoning models, datasets, and application domains to assess its generalisability. Exploring richer trace representations, including semantic and structural features, may further improve uncertainty estimation performance.

Another important direction is investigating practical deployment considerations such as computational cost, latency, scalability, and robustness. Future research may also integrate trace-based uncertainty estimates with selective prediction and abstention mechanisms, enabling models to defer, abstain, or request additional information when uncertainty is high.

Finally, further study is needed to understand the relationship between reasoning trace quality, faithfulness, and uncertainty estimation, which may lead to more reliable and theoretically grounded approaches.

Acknowledgements

I would like to express my sincere gratitude to God for His guidance and blessings throughout my academic journey.

I am grateful to the African Institute for Mathematical Sciences (AIMS) and its partners for providing an excellent learning environment and valuable opportunities. I also thank Google DeepMind for its support through the Google DeepMind Scholarship and for the generous cloud computing resources that supported this research.

My sincere thanks go to my supervisor, Dr Umang Bhatt, and my advisor, Kostadin Devedzhiev, for their guidance and support throughout this work. I am also grateful to Prof. Karin-Therese, Prof. Ulrich Paquet, the teaching assistants, and my colleagues for their encouragement and assistance.

Finally, I would like to thank my family for their unwavering support, encouragement, and belief in me throughout this journey.

References

- Ahdritz, G., Qin, T., Vyas, N., Barak, B., and Edelman, B. L. Distinguishing the knowable from the unknowable with language models, 2024. URL <https://arxiv.org/abs/2402.03563>.
- Anthropic. The claude 3 model family: Opus, sonnet, haiku, 2024. URL https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf. Model card.
- Azaria, A. and Mitchell, T. The internal state of an llm knows when it's lying, 2023. URL <https://arxiv.org/abs/2304.13734>.
- Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., and Liu, Z. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2025. URL <https://arxiv.org/abs/2402.03216>.
- Cuadron, A., Li, D., Ma, W., Wang, X., Wang, Y., Zhuang, S., Liu, S., Schroeder, L. G., Xia, T., Mao, H., Thumiger, N., Desai, A., Stoica, I., Klimovic, A., Neubig, G., and Gonzalez, J. E. The danger of overthinking: Examining the reasoning-action dilemma in agentic tasks, 2025. URL <https://arxiv.org/abs/2502.08235>.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Farquhar, S., Kossen, J., Kuhn, L., and Gal, Y. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, Jun 2024. doi: 10.1038/s41586-024-07421-0.
- Google DeepMind. Gemini 3 pro model card, 2025. URL <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf>. Model card.
- He, P., Liu, X., Gao, J., and Chen, W. DeBERTa: Decoding-enhanced bert with disentangled attention, 2021. URL <https://arxiv.org/abs/2006.03654>.
- Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. The curious case of neural text degeneration, 2020. URL <https://arxiv.org/abs/1904.09751>.
- Hou, B., Liu, Y., Qian, K., Andreas, J., Chang, S., and Zhang, Y. Decomposing uncertainty for large language models through input clarification ensembling, 2024. URL <https://arxiv.org/abs/2311.08718>.
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.
- Hüllermeier, E. and Waegeman, W. Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods. *Machine Learning*, 110(3):457–506, Mar. 2021. ISSN 1573-0565. doi: 10.1007/s10994-021-05946-3. URL <http://dx.doi.org/10.1007/s10994-021-05946-3>.
- Jin, D., Pan, E., Oufattole, N., Weng, W.-H., Fang, H., and Szolovits, P. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14):6421, 2021.

- Joshi, M., Choi, E., Weld, D., and Zettlemoyer, L. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In Barzilay, R. and Kan, M.-Y., editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147/>.
- Kadavath, S., Conerly, T., Aspell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E., Johnston, S., El-Showk, S., Jones, A., Elhage, N., Hume, T., Chen, A., Bai, Y., Bowman, S., Fort, S., Ganguli, D., Hernandez, D., Jacobson, J., Kernion, J., Kravec, S., Lovitt, L., Ndousse, K., Olsson, C., Ringer, S., Amodei, D., Brown, T., Clark, J., Joseph, N., Mann, B., McCandlish, S., Olah, C., and Kaplan, J. Language models (mostly) know what they know, 2022. URL <https://arxiv.org/abs/2207.05221>.
- Kirichenko, P., Ibrahim, M., Chaudhuri, K., and Bell, S. J. Abstentionbench: Reasoning llms fail on unanswerable questions, 2025. URL <https://arxiv.org/abs/2506.09038>.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. Large language models are zero-shot reasoners, 2022.
- Kuhn, L., Gal, Y., and Farquhar, S. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation, 2023. URL <https://arxiv.org/abs/2302.09664>.
- Lin, S., Hilton, J., and Evans, O. Teaching models to express their uncertainty in words, 2022. URL <https://arxiv.org/abs/2205.14334>.
- Mei, Z., Zhang, C., Yin, T., Lidard, J., Shorinwa, O., and Majumdar, A. Reasoning about uncertainty: Do reasoning models know when they don’t know?, 2025. URL <https://arxiv.org/abs/2506.18183>.
- Meta. Llama 3.1 8b instruct. <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>, 2024.
- Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E., and Hashimoto, T. s1: Simple test-time scaling, 2025. URL <https://arxiv.org/abs/2501.19393>.
- OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023. arXiv preprint arXiv:2303.08774.
- OpenAI. Introducing openai o1-preview. <https://openai.com/index/introducing-openai-o1-preview/>, Sept. 2024. Accessed: 2026-06-14.
- Qwen. Qwen3 technical report, 2025a. URL <https://arxiv.org/abs/2505.09388>.
- Qwen. Qwq-32b: Embracing the power of reinforcement learning, March 2025b. URL <https://qwenlm.github.io/blog/qwq-32b/>.
- Shorinwa, O., Mei, Z., Lidard, J., Ren, A. Z., and Majumdar, A. A survey on uncertainty quantification of large language models: Taxonomy, open research challenges, and future directions, 2025. URL <https://arxiv.org/abs/2412.05563>.
- Su, W., Wang, C., Ai, Q., HU, Y., Wu, Z., Zhou, Y., and Liu, Y. Unsupervised real-time hallucination detection based on the internal states of large language models, 2024. URL <https://arxiv.org/abs/2403.06448>.

- Tian, K., Mitchell, E., Zhou, A., Sharma, A., Rafailov, R., Yao, H., Finn, C., and Manning, C. D. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback, 2023. URL <https://arxiv.org/abs/2305.14975>.
- Tomani, C., Chaudhuri, K., Evtimov, I., Cremers, D., and Ibrahim, M. Uncertainty-based abstention in llms improves safety and reduces hallucinations, 2024. URL <https://arxiv.org/abs/2404.10960>.
- Ulinski, M. and Hirschberg, J. Crowdsourced hedge term disambiguation. In Friedrich, A., Zeyrek, D., and Hoek, J., editors, *Proceedings of the 13th Linguistic Annotation Workshop*, pages 1–5, Florence, Italy, Aug. 2019. Association for Computational Linguistics. doi: 10.18653/v1/W19-4001. URL <https://aclanthology.org/W19-4001/>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL <http://arxiv.org/abs/1706.03762>.
- Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E. H., Le, Q., and Zhou, D. Chain of thought prompting elicits reasoning in large language models. *CoRR*, abs/2201.11903, 2022. URL <https://arxiv.org/abs/2201.11903>.
- Welleck, S., Kulikov, I., Roller, S., Dinan, E., Cho, K., and Weston, J. Neural text generation with unlikelihood training, 2019. URL <https://arxiv.org/abs/1908.04319>.
- Wen, B., Yao, J., Feng, S., Xu, C., Tsvetkov, Y., Howe, B., and Wang, L. L. Know your limits: A survey of abstention in large language models, 2025. URL <https://arxiv.org/abs/2407.18418>.
- Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Zhang, B. and Zhang, R. Cot-uq: Improving response-wise uncertainty quantification in llms with chain-of-thought, 2025. URL <https://arxiv.org/abs/2502.17214>.

Appendix A. Data Generation Details

This appendix provides the technical specifications, generation hyperparameters, dataset profiles, and model-specific adapter configurations utilized during the data generation phase of our pipeline. All runs were conducted under uniform environments to ensure consistency across the evaluated reasoning models and non-reasoning control models.

A.1 Hardware and Inference Infrastructure

Parameter	Value	Implementation Details
GPU Hardware	1× NVIDIA A100 40GB	GCP a2-highgpu-1g (2× GPUs for tensor-parallel models)
Inference Engine	vLLM (v0.22.0)	Configured for continuous batching and logprob extraction
Precision Format	BF16	Default loading format for model weights
Chunk Size	50	Concurrent batch size used for pipeline generation passes
Logprob Storage	Summary Stats	Saves mean/max entropy

Table A.1: Hardware and inference infrastructure configurations used during data generation.

A.2 Generation Parameters

Pipeline Pass	Temperature (T)	Top- p	Samples (n)	Max Tokens	Logprobs (k)
Greedy Answer	0.0	—	1	6144	Top-5
Uncertainty Samples	0.7	0.95	10	6144	Top-5
Verbalized Confidence	0.0	—	1	6144	Top-5
$P(\text{True})$ Judgment	0.0	—	1	6144	Top-20

Table A.2: Sampling and generation parameter settings across pipeline passes.

A.3 Dataset Profiles

Dataset	Format	Target Format Instruction	Correctness Labeling Criterion
MedQA	MCQ (5 choices)	“Answer: X” on final line	Exact letter match (A–E)
MMLU-Pro	MCQ (up to 10 choices)	“Answer: X” on final line	Exact letter match (A–J)
TriviaQA	Free-answer	Output inside <code><answer>...</answer></code> tags	Normalized SQuAD/TriviaQA alias match

Table A.3: Dataset evaluation profiles and output formatting requirements.

A.4 Model Specific Configurations

Model Identifier	Type	Split Strategy	Template Kwargs
DeepSeek-R1-Distill-Llama-8B	Reasoning	True (<think>\n)	None
Qwen3-4B	Reasoning	think_tags	enable_thinking: True
Qwen3-4B:no-think	Non-reasoning	inline_cot	enable_thinking: False
Llama-3.1-8B-Instruct	Non-reasoning	inline_cot	None
QwQ-32B	Reasoning	inline_cot	None

Table A.4: Model-specific adapter configurations and chain-of-thought split strategies.

Appendix B. Prompt Templates

This section details the exact prompt templates used for generation, confidence elicitation, and joint probability evaluation ($P(\text{True})$). The placeholders [Question], [Option A], [Proposed Option], and [Proposed Answer] are populated dynamically at runtime.

B.1 Multiple-Choice Datasets (MedQA, MMLU-Pro)

Main Generation Pass (Reasoning Models): For reasoning models, the standard task instruction is prepended.

```
You are answering a multiple-choice question. Reason carefully, then give your final answer as a single letter (one of the options below) on the last line in the form 'Answer: X'.  
Question: [Question]  
A. [Option A]  
B. [Option B]  
...
```

Main Generation Pass (Non-Reasoning Control Models): For non-reasoning control models, a zero-shot Chain-of-Thought (CoT) instruction is prepended instead.

```
Answer the following multiple-choice question. Let's think step by step, then give your final answer as a single letter on the last line in the form 'Answer: X'.  
Question: [Question]  
A. [Option A]  
B. [Option B]  
...
```

Verbalized Confidence Pass: This prompt collects the model's confidence estimation following its greedy generation.

```
Question: [Question]  
A. [Option A]  
B. [Option B]  
...  
Your proposed answer: [Proposed Option Letter]  
How confident are you, on a scale of 0 to 100, that your proposed answer is correct?  
Reply with only the integer score on the final line in the form 'Confidence: NN'.
```

P(True) Pass: This prompt is evaluated under a reason-then-judge design to read token probability distribution at the final validation index.

```

Question: [Question]
A. [Option A]
B. [Option B]
...
Proposed answer: [Proposed Option Letter]. [Proposed Option Text]
Is the proposed answer correct? Reason through it, then on the final line answer with a
single word, exactly 'True' or 'False' (no letter prefix, no punctuation).

```

B.2 Free-Answer Dataset (TriviaQA)

Main Generation Pass (Greedy and Samples): TriviaQA outputs are framed using structural tags to decouple reasoning length from extraction processing.

```

Answer the following question. Reason freely first if you need to. At the very end of
your response, output your final short answer inside <answer>...</answer> tags. Put
nothing else inside those tags.
Question: [Question]

```

Verbalized Confidence Pass: This configuration evaluates verbalized confidence on free-answer formulations.

```

Question: [Question]
Your proposed answer: [Proposed Answer]
How confident are you, on a scale of 0 to 100, that your proposed answer is correct?
Reply with only the integer score on the final line in the form 'Confidence: NN'.

```

P(True) Pass: The joint-probability pass for the short-answer prediction.

```

Question: [Question]
Proposed answer: [Proposed Answer]
Is the proposed answer correct? Reason through it, then on the final line answer with a
single word, exactly 'True' or 'False' (no letter prefix, no punctuation).

```


Appendix C. Results

C.1 Single-Feature AUROC Across All Datasets and Models

Dataset	Model	Trace Length	Trace Divergence	Rep 5	Hedging Formal	Hedging Reasoning	Connector Density
medqa	llama-3.1-8b-instruct	0.612	0.594	0.508	0.549	0.533	0.526
	qwen3-4b	0.750	0.519	0.755	0.684	0.628	0.546
	qwen3-4b-nothink	0.597	0.606	0.579	0.554	0.540	0.542
	r1-distill-llama-8b	0.660	0.583	0.608	0.616	0.575	0.546
mmlu_pro	llama-3.1-8b-instruct	0.577	0.609	0.529	0.570	0.513	0.544
	qwen3-4b	0.666	0.619	0.772	0.669	0.603	0.590
	qwen3-4b-nothink	0.617	0.643	0.615	0.586	0.597	0.566
	qwq-32b	0.602	0.634	0.683	0.661	0.585	0.518
	r1-distill-llama-8b	0.650	0.589	0.661	0.555	0.551	0.502
trivia_qa	llama-3.1-8b-instruct	0.557	0.626	0.574	0.541	0.542	0.534
	qwen3-4b	0.774	0.779	0.800	0.650	0.768	0.750
	qwen3-4b-nothink	0.647	0.794	0.622	0.587	0.568	0.562
	qwq-32b	0.732	0.708	0.704	0.663	0.724	0.599
	r1-distill-llama-8b	0.684	0.718	0.619	0.607	0.622	0.632

Table C.1: AUROC scores for individual features across datasets and models. The strongest feature per row is highlighted in bold.

C.2 Trace-LR vs. Baselines: AURC

Dataset	Model	Trace LR		Semantic Entropy		p_{true}		Verbalized Confidence	
		AURC ↓	Acc@80% ↑	AURC ↓	Acc@80% ↑	AURC ↓	Acc@80% ↑	AURC ↓	Acc@80% ↑
medqa	llama-3.1-8b-instruct	0.2454	0.7047	0.1708	0.7304	0.3192	0.6671	0.2890	0.6658
	qwen3-4b	0.0976	0.8277	0.1675	0.8361	0.1753	0.7753	0.1556	0.7912
	qwen3-4b-nothink	0.2428	0.6842	0.2297	0.7018	0.3015	0.6937	0.2872	0.6667
	r1-distill-llama-8b	0.3324	0.5599	0.3293	0.5741	0.4174	0.5478	0.4235	0.5463
mmlu_pro	llama-3.1-8b-instruct	0.3425	0.5846	0.2577	0.6103	0.4085	0.5718	0.3980	0.5841
	qwen3-4b	0.0924	0.8305	0.1716	0.8425	0.1954	0.7719	0.1494	0.7915
	qwen3-4b-nothink	0.2096	0.7107	0.2062	0.7438	0.2829	0.7019	0.3005	0.7050
	qwq-32b	0.0403	0.9411	0.0813	0.9278	0.0630	0.9049	0.0688	0.9185
trivia_qa	r1-distill-llama-8b	0.3596	0.5678	0.3326	0.5649	0.3916	0.5729	0.4282	0.5106
	llama-3.1-8b-instruct	0.2994	0.6896	0.1944	0.7385	0.2948	0.6959	0.3477	0.6570
	qwen3-4b	0.2860	0.5861	0.2613	0.6025	0.3071	0.5779	0.3490	0.5907
	qwen3-4b-nothink	0.3720	0.5032	0.3621	0.5171	0.4403	0.5019	0.5137	0.4899
	qwq-32b	0.1549	0.8105	0.1381	0.8379	0.1745	0.7843	0.1680	0.7822
	r1-distill-llama-8b	0.4460	0.4450	0.3847	0.4728	0.4817	0.4324	0.5083	0.4350

Table C.2: AURC and accuracy at 80% coverage (Acc@80%) across datasets and models. For each row, the strongest AURC (lowest) and Acc@80% (highest) are highlighted in bold.

C.3 Trace-LR vs. Baselines: AUROC

Dataset	Model	Trace LR	Semantic Entropy	p_{true}	Verbalized Confidence
medqa	llama-3.1-8b-instruct	0.654	0.784	0.550	0.555
	qwen3-4b	0.777	0.683	0.595	0.641
	qwen3-4b-nothink	0.653	0.696	0.621	0.593
	r1-distill-llama-8b	0.670	0.684	0.585	0.564
mmlu_pro	llama-3.1-8b-instruct	0.656	0.778	0.590	0.605
	qwen3-4b	0.818	0.723	0.602	0.695
	qwen3-4b-nothink	0.691	0.761	0.617	0.626
	qwq-32b	0.744	0.596	0.609	0.629
	r1-distill-llama-8b	0.698	0.728	0.629	0.561
trivia_qa	llama-3.1-8b-instruct	0.648	0.791	0.616	0.562
	qwen3-4b	0.827	0.871	0.780	0.743
	qwen3-4b-nothink	0.796	0.843	0.715	0.616
	qwq-32b	0.759	0.807	0.715	0.715
	r1-distill-llama-8b	0.733	0.834	0.686	0.656

Table C.3: AUROC scores for Trace LR versus baseline methods across datasets and models. The strongest method per row is highlighted in bold.

Appendix D. Calibration and Bootstrap Comparison Results

D.1 Trace LR vs. Semantic Entropy: Point Estimates

Dataset	Model	n	Brier Score		NLL		ECE	
			Trace LR	Sem. Entropy	Trace LR	Sem. Entropy	Trace LR	Sem. Entropy
medqa	qwen3-4b	740	0.155	0.1596	0.4702	0.4946	0.0551	0.0356
	r1-distill-llama-8b	792	0.2282	0.2235	0.6474	0.638	0.0447	0.054
	qwen3-4b-nothink	926	0.2171	0.205	0.6224	0.598	0.0584	0.0343
	llama-3.1-8b-instruct	974	0.2117	0.1776	0.6124	0.5299	0.0239	0.047
mmlu_pro	qwen3-4b	730	0.146	0.1465	0.4468	0.4657	0.0538	0.0125
	r1-distill-llama-8b	598	0.2216	0.2126	0.6361	0.6151	0.0345	0.0364
	qwq-32b	657	0.0807	0.0827	0.2832	0.2999	0.0162	0.0145
	qwen3-4b-nothink	908	0.2041	0.1784	0.5935	0.538	0.0344	0.0433
	llama-3.1-8b-instruct	731	0.2303	0.191	0.6529	0.5674	0.0121	0.0436
trivia_qa	qwen3-4b	915	0.1672	0.1387	0.5162	0.4409	0.0496	0.0434
	r1-distill-llama-8b	944	0.1993	0.1602	0.5817	0.4839	0.0229	0.0353
	qwq-32b	956	0.1565	0.1337	0.4859	0.4292	0.0246	0.0288
	qwen3-4b-nothink	989	0.1806	0.1516	0.5416	0.4613	0.0146	0.0256
	llama-3.1-8b-instruct	818	0.2158	0.1712	0.6255	0.5189	0.0427	0.034

Table D.1: Point estimates for Brier score, negative log-likelihood (NLL), and expected calibration error (ECE) under Trace LR versus Semantic Entropy (Platt-scaled).

D.2 Trace LR vs. Semantic Entropy: Bootstrap Comparison

Dataset	Model	n	Brier Score ($\Delta_{SE-Trace}$)			NLL ($\Delta_{SE-Trace}$)		
			Δ	95% CI	Trace Better (%)	Δ	95% CI	Trace Better (%)
medqa	qwen3-4b	740	0.0046	[-0.0058, 0.0148]	78.7	0.0241	[-0.0026, 0.0498]	95.6
	r1-distill-llama-8b	792	-0.0049	[-0.015, 0.0064]	20.0	-0.0094	[-0.0313, 0.0147]	22.1
	qwen3-4b-nothink	926	-0.0122	[-0.0217, -0.0018]	1.2	-0.0247	[-0.0465, -0.0026]	1.9
	llama-3.1-8b-instruct	974	-0.0344	[-0.0457, -0.0233]	0.0	-0.0828	[-0.1102, -0.0563]	0.0
mmlu_pro	qwen3-4b	730	0.0008	[-0.0113, 0.0134]	54.6	0.02	[-0.0132, 0.051]	88.0
	r1-distill-llama-8b	598	-0.0093	[-0.0238, 0.0054]	11.0	-0.0215	[-0.056, 0.0116]	11.0
	qwq-32b	657	0.0019	[-0.0036, 0.0075]	75.2	0.0165	[-0.0029, 0.0374]	94.8
	qwen3-4b-nothink	908	-0.0256	[-0.0373, -0.0132]	0.0	-0.0552	[-0.0825, -0.0262]	0.0
	llama-3.1-8b-instruct	731	-0.0388	[-0.0539, -0.0238]	0.0	-0.0848	[-0.1188, -0.0493]	0.0
trivia_qa	qwen3-4b	915	-0.0283	[-0.04, -0.0177]	0.0	-0.0742	[-0.1039, -0.047]	0.0
	r1-distill-llama-8b	944	-0.0391	[-0.0496, -0.0288]	0.0	-0.0976	[-0.1244, -0.0725]	0.0
	qwq-32b	956	-0.0231	[-0.032, -0.0134]	0.0	-0.0573	[-0.0802, -0.0316]	0.0
	qwen3-4b-nothink	989	-0.029	[-0.0392, -0.0183]	0.0	-0.0799	[-0.1078, -0.053]	0.0
	llama-3.1-8b-instruct	818	-0.0445	[-0.0569, -0.0321]	0.0	-0.1064	[-0.1362, -0.0779]	0.0

Table D.2: Paired bootstrap comparison of Trace LR versus Semantic Entropy (Platt-scaled) on Brier score and NLL.

D.3 Calibration Error by Method

Dataset	Model	Method	n	ECE (Raw)	Mean Confidence	Accuracy
medqa	qwen3-4b	p_{true}	740	0.2347	0.9878	0.7635
		Verbalized Confidence	736	0.2005	0.9536	0.7622
		Semantic Entropy	740	0.1626	0.9236	0.7635
	r1-distill-llama-8b	p_{true}	785	0.4464	0.9582	0.5236
		Verbalized Confidence	782	0.3726	0.893	0.5205
		Semantic Entropy	792	0.1246	0.6169	0.5215
	qwen3-4b-nothink	p_{true}	926	0.3302	0.9037	0.6415
		Verbalized Confidence	926	0.3175	0.9587	0.6415
		Semantic Entropy	926	0.1594	0.7874	0.6415
	llama-3.1-8b-instruct	p_{true}	973	0.3271	0.9594	0.6506
		Verbalized Confidence	973	0.1845	0.8197	0.6516
		Semantic Entropy	974	0.0824	0.7025	0.6509
mmlu_pro	qwen3-4b	p_{true}	712	0.3024	0.7926	0.7542
		Verbalized Confidence	708	0.2373	0.9419	0.7571
		Semantic Entropy	730	0.1922	0.9428	0.7507
	r1-distill-llama-8b	p_{true}	480	0.3847	0.7776	0.5312
		Verbalized Confidence	587	0.4023	0.8321	0.5026
		Semantic Entropy	598	0.2078	0.7028	0.495
	qwq-32b	p_{true}	657	0.097	0.9763	0.9041
		Verbalized Confidence	629	0.0799	0.9455	0.903
		Semantic Entropy	657	0.0856	0.9877	0.9041
	qwen3-4b-nothink	p_{true}	897	0.3331	0.79	0.66
		Verbalized Confidence	903	0.3174	0.9695	0.6589
		Semantic Entropy	908	0.1887	0.8473	0.6586
	llama-3.1-8b-instruct	p_{true}	531	0.3791	0.8387	0.5386
		Verbalized Confidence	721	0.3721	0.8942	0.5381
		Semantic Entropy	731	0.2018	0.7394	0.5376
trivia_qa	qwen3-4b	p_{true}	915	0.4379	0.8951	0.4918
		Verbalized Confidence	910	0.4421	0.9261	0.4945
		Semantic Entropy	915	0.2166	0.7084	0.4918
	r1-distill-llama-8b	p_{true}	942	0.4097	0.6757	0.3822
		Verbalized Confidence	943	0.4387	0.8205	0.3818
		Semantic Entropy	944	0.1406	0.522	0.3814
	qwq-32b	p_{true}	956	0.2598	0.9504	0.7144
		Verbalized Confidence	953	0.2422	0.9589	0.7167
		Semantic Entropy	956	0.14	0.8536	0.7144
	qwen3-4b-nothink	p_{true}	989	0.4	0.7803	0.4186
		Verbalized Confidence	987	0.4776	0.8767	0.4195
		Semantic Entropy	989	0.2971	0.7157	0.4186
	llama-3.1-8b-instruct	p_{true}	769	0.3341	0.9328	0.6411
		Verbalized Confidence	816	0.3479	0.9367	0.625
		Semantic Entropy	818	0.0837	0.6861	0.6259

Table D.3: Calibration error (ECE), mean confidence, and accuracy for each confidence-elicitation method, broken down by dataset and model.